# 11 — Fehlerbehandlung

Programme mit try / except / finally robust machen

HTW Berlin – Wirtschaftsingenieurwesen

WiSe 2026/27

# Agenda

1. Wozu Fehlerbehandlung?
2. Was ist eine **Exception**? Tracebacks lesen
3. `try` / `except` – Fehler fangen
4. **Spezifisch** fangen & die Fehlerklassen-Landkarte
5. `finally` – Aufräumen, garantiert
6. EVA-Perspektive: Eingaben validieren
7. **Klassiker: Die unzerstörbare Eingabe** – live programmiert

**Lernziel**: Programme bauen, die bei falschen Eingaben nicht sterben – und **verstehen**, was bei einem Crash wirklich passiert.

**Heute**: nach jedem Konzept *Live-Demo* (ich) und *Sofort ausprobieren* (Sie, Notebook 11).

# Wozu Fehlerbehandlung?

## Ohne Behandlung

```
menge = int(input("Bestellmenge: "))
# Kunde tippt "drei" -> ValueError,
# der Shop bricht ab, Bestellung weg
```

## Mit Behandlung

```
try:
    menge = int(input("Bestellmenge: "))
except ValueError:
    print("Bitte eine ganze Zahl!")
```

**Robust** = das Programm bleibt handlungsfähig, auch wenn von außen Unsinn kommt. Ein Shop, der bei jedem Tippfehler abstürzt, verkauft keine Sneaker.

# Was ist eine Exception?

```
menge = int("drei")        # hier knallt es
```

- Läuft etwas schief, erzeugt Python an der Fehlerstelle ein **Ausnahme-Objekt** (Exception) – mit **Typ** (`ValueError`) und **Botschaft**
- Die normale Ausführung **stoppt sofort** – der Rest des Programms läuft erst mal nicht weiter
- **Ungefangen** wandert das Objekt bis ganz nach oben → Python druckt den **Traceback** und beendet das Programm
- **Gefangen** (heute neu): *wir* entscheiden, wie es weitergeht

# Unter der Haube: ein Crash im Zeitraffer

```
print("Willkommen bei Veggie Soles")   # Z1
menge = int("drei")                    # Z2
print("Bestellung:", menge)            # Z3
```

| Schritt | Python tut | Folge |
|---|---|---|
| 1 | Z1: `print` läuft | Ausgabe erscheint |
| 2 | Z2: `int("drei")` scheitert | `ValueError`-Objekt entsteht |
| 3 | normale Ausführung stoppt | niemand fängt das Objekt |
| 4 | Objekt erreicht oberste Ebene | Traceback – Z3 läuft **nie** |

Eine Exception ist ein **Objekt**, kein Zufallstext: Typ + Botschaft entstehen exakt an der Fehlerstelle.

# Unter der Haube: Tracebacks lesen

```
Traceback (most recent call last):
  File "shop.py", line 2, in <module>
    menge = int("drei")
ValueError: invalid literal for int()
            with base 10: 'drei'
```

- **Von unten nach oben lesen!**
- Letzte Zeile: **Typ** (`ValueError`) + **Botschaft** (*was* war falsch)
- Darüber: **wo** – Datei, Zeilennummer, die Codezeile selbst

**Faustregel**: erst die letzte Zeile verstehen, dann zur genannten Zeile springen – das ist 90 % der Fehlersuche.

# try / except – die Grundstruktur

```
try:
    menge = int(eingabe)        # riskant
    print("OK:", menge)
except ValueError:
    print("Bitte eine ganze Zahl!")
```

| Block | Wann läuft er? |
|---|---|
| `try:` | immer – bis zum ersten Fehler |
| `except ValueError:` | nur wenn im `try` ein `ValueError` fliegt |

- Kein Fehler → `except` wird komplett **übersprungen**
- Nach beiden Wegen läuft das Programm **normal weiter**

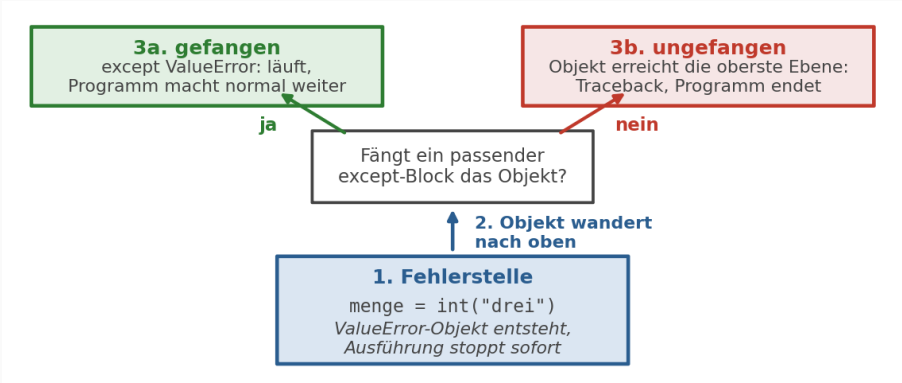
# Unter der Haube: `try/except` als Sprung

```
try:
    menge = int(eingabe)          # Z2
    print("OK:", menge)           # Z3
except ValueError:
    print("Keine ganze Zahl.")    # Z5
print("weiter")                   # Z6
```

| Fall | Z2 | Z3 | Z5 | Z6 |
|---|---|---|---|---|
| `eingabe = "3"` | klappt | läuft | übersprungen | läuft |
| `eingabe = "drei"` | `ValueError` | läuft **nie** | fängt | läuft |

Fliegt die Ausnahme, springt die Ausführung **sofort** in den passenden `except`-Block – der Rest des `try`-Blocks wird **nie** ausgeführt. Zwei Fälle, wie beim Short-Circuit.

# Unter der Haube: der Weg einer Ausnahme




`try/except` ändert nicht, **dass** der Fehler entsteht – es entscheidet nur, **wer** das Objekt auffängt, bevor es oben ankommt.

# try / except – Demo & Übung

▶ **Live-Demo** – `01_try_except_einfach.py`

✎ **Sofort ausprobieren** – Notebook 11, Angeleitete Übung 1.1: Division mit Eingabe absichern

# Spezifisch fangen: `except ValueError` & Co.

```
try:
    anzahl = int(text)          # ValueError?
    preis = preise[modell]      # KeyError?
except ValueError:
    print("Bitte eine ganze Zahl.")
except KeyError:
    print("Modell nicht im Sortiment.")
```

- Python prüft die except-Blöcke **von oben nach unten** – der erste passende gewinnt
- Mehrere Typen, gleiche Reaktion: `except (ValueError, KeyError):`
- Botschaft mitlesen: `except ValueError as e: print(e)`

# Unter der Haube: warum spezifisch fangen?

```
try:
    zahl = int(einggabe)      # Tippfehler!
except:
    print("Keine Zahl!")      # falsche Diagnose
```

- Nacktes `except:` fängt **alles** – auch den `NameError` aus dem Tippfehler `einggabe`
- Der Nutzer liest “Keine Zahl!”, obwohl seine Eingabe korrekt war – der **echte Bug bleibt unsichtbar**
- Mit `except ValueError:` crasht der Tippfehler ehrlich mit Traceback → in Sekunden gefunden

Fangen Sie nur, was Sie **erwarten** und behandeln **können** – alles andere darf laut crashen.

# Die Fehlerklassen-Landkarte

| Exception | Wann fliegt sie? | Mini-Auslöser |
|---|---|---|
| `ValueError` | richtiger Typ, unpassender Wert | `int("drei")` |
| `TypeError` | Operation auf falschem Typ | `"Preis: " + 5` |
| `ZeroDivisionError` | Division durch null | `100 / 0` |
| `KeyError` | Dict-Schlüssel fehlt | `preise["Sandale"]` |
| `IndexError` | Listen-Index zu groß | `korb[99]` |

Die letzte Traceback-Zeile nennt Ihnen genau den Typ, den Ihr `except` fangen muss – Landkarte lesen statt raten.

# Spezifisch fangen – Demo & Übung

▶ **Live-Demo** – `02_spezifische_exceptions.py`

✎ **Sofort ausprobieren** – Notebook 11, Angeleitete Übung 2.1: Kreisumfang mit `ValueError` absichern

# `finally` – Aufräumen, garantiert

```
try:
    anzahl = int(eingabe)
    print("Buche", anzahl, "Paar.")
except ValueError:
    print("Keine ganze Zahl.")
finally:
    print("Kasse abgeschlossen.")
```

- `finally` läuft **immer** – mit Fehler, ohne Fehler, sogar bei ungefangenen Fehlern
- Typisch: Statusmeldung, Zähler, später Ressourcen freigeben

# Unter der Haube: `finally` läuft IMMER

| Schritt | Fall A: `"3"` | Fall B: `"drei"` |
|---|---|---|
| `try` | läuft komplett | stoppt bei `int()` |
| `except` | übersprungen | fängt, Hinweis |
| `finally` | **läuft** | **läuft** |
| danach | weiter | weiter |

- Beide Fälle enden im selben `finally` – deshalb heißt es *Aufräum-Garantie*
- Ausblick: in Notebook 12 räumen wir damit **Dateien** auf – geöffnete Ressourcen müssen auch im Fehlerfall geschlossen werden

Merkreihenfolge: `try` → ggf. `except` → **immer** `finally` → Code danach.

# `finally` – Demo & Übung

▶ **Live-Demo** – `03_finally_block.py`

✎ **Sofort ausprobieren** – Notebook 11, Angeleitete Übung 3.1: try/except/finally mit Statusmeldung

# EVA-Perspektive: Eingaben sind Feindesland

- **EVA** (seit E4): **E**ingabe → **V**erarbeitung → **A**usgabe – Laufzeitfehler entstehen fast immer beim **E**
- Alles von außen kann kaputt sein: Tippfehler, Kommazahlen, leere Eingaben – heute `input()`, später Dateien und Netz
- **An der Grenze validieren**: `try`/`except` direkt um die Eingabe – die Verarbeitung rechnet nur noch mit geprüften Werten

▶ **Live-Demo** – `04_eingabe_validieren.py`

✎ **Sofort ausprobieren** – Notebook 11, Angeleitete Übung 1.2: Divisions-Rechner in der Schleife robust machen

# Klassiker: Die unzerstörbare Eingabe – die Aufgabe

Der Veggie-Soles-Shop fragt die **Bestellmenge** ab – und übersteht **jede** Eingabe:

- `drei`, `3.5` → **“Bitte eine ganze Zahl!”**, erneut fragen
- Die erste **ganze Zahl** → Bestätigung ausgeben
- Kernstück: Funktion `lese_ganzzahl(prompt)` (`def`, NB 07)

**So soll es laufen**

```
Bestellmenge: drei
Bitte eine ganze Zahl!
Bestellmenge: 3.5
Bitte eine ganze Zahl!
Bestellmenge: 3
Alles klar: 3 Paar Sneaker.
```

# Klassiker: Die unzerstörbare Eingabe – Live-Coding

Unser Zahlenraten aus Foliensatz 10 stirbt bei abc – heute machen wir es **unsterblich**.

**▶ Live-Coding ab leerem Editor**
Referenz: `90_klassiker_robuste_eingabe.py` – Teilziele:

1. Naive Version crashen lassen: `int("drei")`
2. `try` um die heikle Zeile
3. `except ValueError` mit Hinweis
4. Schleife drumherum + `break` bei Erfolg
5. Als Funktion `lese_ganzzahl(prompt)` wiederverwendbar machen

✎ **Zum Selberlösen** – Handout *"Klassiker 11: Die unzerstörbare Eingabe"* (`Klassiker/11_Klassiker_Robuste_Eingabe.pdf`). Vertiefung: Notebook 11, Abschlussübungen.

# Cheat Card

| Aufgabe | Syntax |
|---|---|
| Fehler fangen | `try: ... except ValueError: ...` |
| Mehrere Typen | `except (ValueError, KeyError):` |
| Botschaft lesen | `except ValueError as e:` |
| Immer ausführen | `finally: ...` |
| Traceback lesen | von unten: letzte Zeile = Typ + Botschaft |
| Klassiker heute | `while True:` + `try`/`break` + `except` |

**Faustregel**: **spezifisch** fangen, **nah** an der Fehlerstelle, hilfreiche Meldung – Fehlerbehandlung ist für den Anwender da, nicht zum Verstecken eigener Bugs.

# Ausblick: Notebook 12 – File I/O

Heute kamen Eingaben von der Tastatur – nächste Woche kommen sie aus **Dateien**: fehlende Pfade, falsche Namen, kaputte Inhalte. EVA-Feindesland pur.

```
# Vorschau -- diese Syntax lernen wir erst
# naechste Woche kennen:
try:
    with open("bestellungen.csv") as f:
        text = f.read()
except FileNotFoundError:
    print("Datei fehlt -- Standardwerte.")
```

`try`/`except` von heute plus `open`/`with` von nächster Woche = das Standard-Muster für robustes Lesen und Schreiben.

# Heute geübt

- ✓ Exception = **Objekt** mit Typ + Botschaft, Ausführung stoppt sofort
- ✓ Tracebacks **von unten nach oben** lesen
- ✓ `try`/`except` als Sprung: Rest des `try`-Blocks läuft nie
- ✓ **Spezifisch** fangen – nacktes `except:` versteckt Bugs
- ✓ Fehlerklassen-Landkarte: `ValueError`, `TypeError`, `KeyError` & Co.
- ✓ `finally` läuft immer – die Aufräum-Garantie
- ✓ **Klassiker Die unzerstörbare Eingabe** live gesehen

✎ **Ihre Aufgaben bis zur nächsten Woche:**

- Klassiker **Die unzerstörbare Eingabe** eigenständig lösen (Handout)
- Bonus: Zahlenraten aus Foliensatz 10 mit `lese_ganzzahl()` härten
- “Sofort ausprobieren”-Aufgaben + Abschlussübungen im Notebook 11